JobSheet - Week 4 - Polymorphism

Nama : **Alia Ardani**
NIM : **251524035**
Kelas : **1B**
Repo GitHub : **https://github.com/vssixla/Teknik_Pemrograman_2026**

Instruksi Pengerjaan:

1. Kerjakan 3 soal di bawah ini dengan melengkapi setiap kolom jawaban yang disediakan pada jobsheet ini.
2. Jawaban setiap soal mencakup source code, screenshot hasil dari program yang ditampilkan full screen termasuk taskbar (tambahkan beberapa screenshot jika diperlukan), penjelasan permasalahan dan solusi yang dihadapi, nama teman yang membantu memecahkan masalah (opsional).
3. Dikumpulkan pada Assignment Classroom sesuai dengan deadline yang tertera pada assignment tersebut.
4. Format penamaan file jobsheet: W4_P_<Kelas 1X>_<3 Digit_NIM_Terakhir>.docx/pdf. Contoh: W3_P_1B_001.docx/pdf.
5. Submit semua jawaban dalam bentuk file java pada repository GitHub masing-masing.

No. 1 Inheritance via Polymorphism (Another Type of Employee)

Soal Praktikum
Silahkan membaca soal pada file lampiran: 1-Polymorphism via Inheritance.pdf
Source Code

Firm.java

```
//*******************************************************************
// Firm.java Author: Lewis/Loftus
//
// Demonstrates polymorphism via inheritance.
//*******************************************************************
public class Firm
{
//-----------------------------------------------------------------
// Creates a staff of employees for a firm and pays them.
//-----------------------------------------------------------------
public static void main (String[] args)
{
Staff personnel = new Staff();
personnel.payday();
}
}
```

Staff.java

```
//*******************************************************************
// Staff.java Author: Lewis/Loftus
//
// Represents the personnel staff of a particular business.
//*******************************************************************
public class Staff
{
   StaffMember[] staffList;
   //-----------------------------------------------------------------
   // Sets up the list of staff members.
   //-----------------------------------------------------------------
   public Staff ()
   {
      staffList = new StaffMember[8]; //10. Mengubah ukuran staff [8]
      staffList[0] = new Executive ("Sam", "123 Main Line",
                  "555-0469", "123-45-6789", 2423.07);
      staffList[1] = new Employee ("Carla", "456 Off Line",
                  "555-0101", "987-65-4321", 1246.15);
      staffList[2] = new Employee ("Woody", "789 Off Rocker",
                  "555-0000", "010-20-3040", 1169.23);
      staffList[3] = new Hourly ("Diane", "678 Fifth Ave.",
                  "555-0690", "958-47-3625", 10.55);
      staffList[4] = new Volunteer ("Norm", "987 Suds Blvd.",
```

```
                "555-8374");
    staffList[5] = new Volunteer ("Cliff", "321 Duds Lane",
                "555-7282");
    ((Executive)staffList[0]).awardBonus (500.00);
    ((Hourly)staffList[3]).addHours (40);

    //11. Menambahkan 2 Commission Employee
    staffList[6] = new Commission("Ali", "Jalan Mawar", "555-2222",
            "111-22-3333", 6.25, 0.20);
            //12. Data employee 1 : Hourly 6.25, Commision 20%
    staffList[7] = new Commission("Budi", "Jalan Melati", "555-3333",
            //12. Data employee 2 : Hourly 9.75, Commision 15%
            "222-33-4444", 9.75, 0.15);

    //13. Menambahkan Hours dan Sales
    ((Commission)staffList[6]).addHours(35);
    ((Commission)staffList[6]).addSales(400);

    ((Commission)staffList[7]).addHours(40);
    ((Commission)staffList[7]).addSales(950);
  }
  //-----------------------------------------------------------------
  // Pays all staff members.
  //-----------------------------------------------------------------
  public void payday ()
  {
    double amount;
    for (int count=0; count < staffList.length; count++)
  {
    System.out.println (staffList[count]);
    amount = staffList[count].pay(); // polymorphic
    if (amount == 0.0)
      System.out.println ("Thanks!");
    else
      System.out.println ("Paid: " + amount);
      System.out.println ("-----------------------------------");
    }
  }
}
```

StaffMember.java

```
//*******************************************************************
// StaffMember.java Author: Lewis/Loftus
//
// Represents a generic staff member.
//*******************************************************************
abstract public class StaffMember
{
protected String name;
```

```
protected String address;
protected String phone;
//-----------------------------------------------------------------
// Sets up a staff member using the specified information.
//-----------------------------------------------------------------
public StaffMember (String eName, String eAddress, String ePhone)
{
name = eName;
address = eAddress;
phone = ePhone;
}
//-----------------------------------------------------------------
// Returns a string including the basic employee information.
//-----------------------------------------------------------------
public String toString()
{
String result = "Name: " + name + "\n";
result += "Address: " + address + "\n";
result += "Phone: " + phone;
return result;
}
//-----------------------------------------------------------------
// Derived classes must define the pay method for each type of
// employee.
//-----------------------------------------------------------------
public abstract double pay();
}
```

Volunteer.java

```
//*******************************************************************
// Volunteer.java Author: Lewis/Loftus
//
// Represents a staff member that works as a volunteer.
//*******************************************************************
public class Volunteer extends StaffMember
{
   //-----------------------------------------------------------------
   // Sets up a volunteer using the specified information.
   //-----------------------------------------------------------------
   public Volunteer (String eName, String eAddress, String ePhone)
   {
      super (eName, eAddress, ePhone);
   }
   //-----------------------------------------------------------------
   // Returns a zero pay value for this volunteer.
   //-----------------------------------------------------------------
   public double pay()
   {
   return 0.0;
```

```
  }
}
```

Employee.java

```
//********************************************************************
// Employee.java Author: Lewis/Loftus
//
// Represents a general paid employee.
//********************************************************************
public class Employee extends StaffMember
{
  protected String socialSecurityNumber;
  protected double payRate;
  //-----------------------------------------------------------------
  // Sets up an employee with the specified information.
  //-----------------------------------------------------------------
  public Employee (String eName, String eAddress, String ePhone,
  String socSecNumber, double rate)
  {
    super (eName, eAddress, ePhone);
    socialSecurityNumber = socSecNumber;
    payRate = rate;
  }
  //-----------------------------------------------------------------
  // Returns information about an employee as a string.
  //-----------------------------------------------------------------
  public String toString()
  {
    String result = super.toString();
    result += "\nSocial Security Number: " + socialSecurityNumber;
    return result;
  }
  //-----------------------------------------------------------------
  // Returns the pay rate for this employee.
  //-----------------------------------------------------------------
  public double pay()
  {
    return payRate;
  }
}
```

Executive.java

```
//********************************************************************
// Executive.java Author: Lewis/Loftus
//
// Represents an executive staff member, who can earn a bonus.
//********************************************************************
public class Executive extends Employee
{
```

```
   private double bonus;
   //-----------------------------------------------------------------
   // Sets up an executive with the specified information.
   //-----------------------------------------------------------------
   public Executive (String eName, String eAddress, String ePhone,
   String socSecNumber, double rate)
   {
      super (eName, eAddress, ePhone, socSecNumber, rate);
      bonus = 0; // bonus has yet to be awarded
   }
   //-----------------------------------------------------------------
   // Awards the specified bonus to this executive.
   //-----------------------------------------------------------------
   public void awardBonus (double execBonus)
   {
      bonus = execBonus;
   }
   //-----------------------------------------------------------------
   // Computes and returns the pay for an executive, which is the
   // regular employee payment plus a one-time bonus.
   //-----------------------------------------------------------------
   public double pay()
   {
      double payment = super.pay() + bonus;
      bonus = 0;
      return payment;
   }
}
```

Hourly.java

```
//********************************************************************
// Hourly.java Author: Lewis/Loftus
//
// Represents an employee that gets paid by the hour.
//********************************************************************
public class Hourly extends Employee
{
   private int hoursWorked;
   //-----------------------------------------------------------------
   // Sets up this hourly employee using the specified information.
   //-----------------------------------------------------------------
   public Hourly (String eName, String eAddress, String ePhone,
   String socSecNumber, double rate)
   {
      super (eName, eAddress, ePhone, socSecNumber, rate);
      hoursWorked = 0;
   }
   //-----------------------------------------------------------------
   // Adds the specified number of hours to this employee's
```

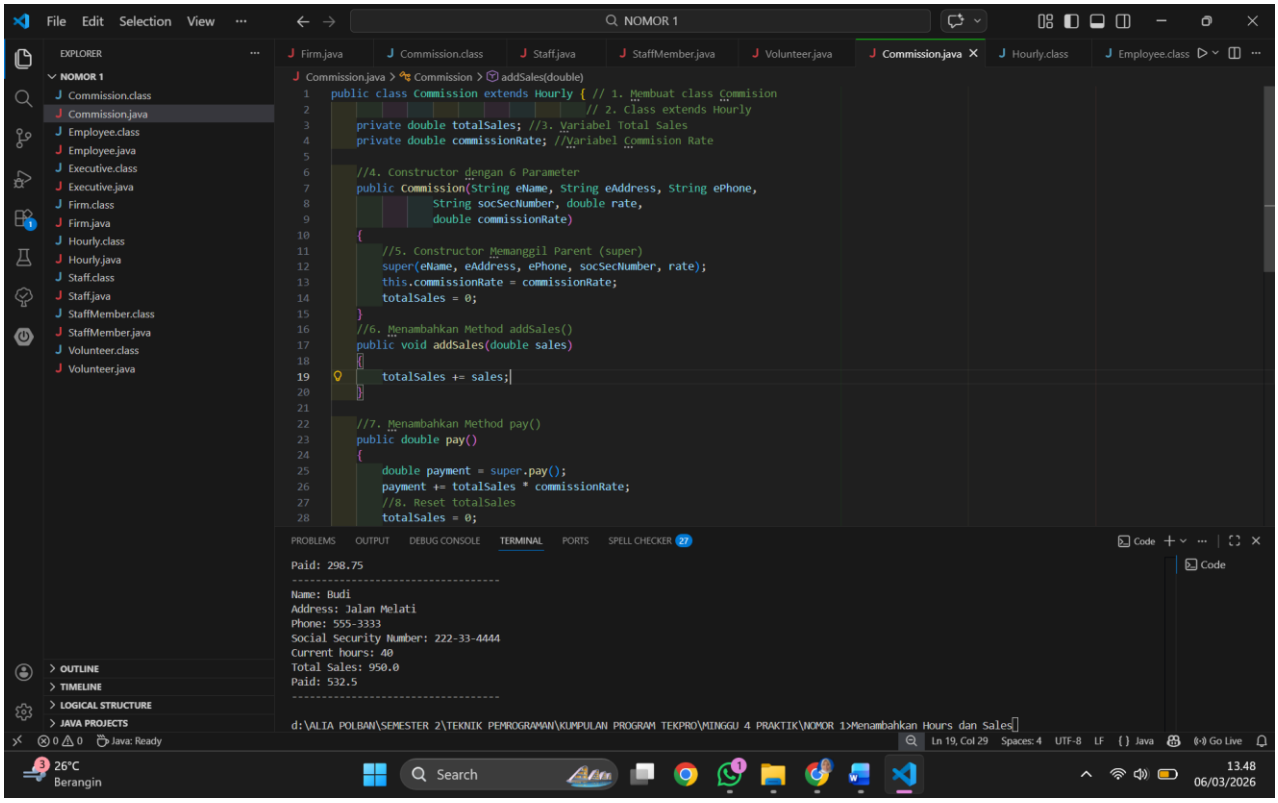
```
  // accumulated hours.
  //-----------------------------------------------------------------
  public void addHours (int moreHours)
  {
    hoursWorked += moreHours;
  }
  //-----------------------------------------------------------------
  // Computes and returns the pay for this hourly employee.
  //-----------------------------------------------------------------
  public double pay()
  {
    double payment = payRate * hoursWorked;
    hoursWorked = 0;
    return payment;
  }
  //-----------------------------------------------------------------
  // Returns information about this hourly employee as a string.
  //-----------------------------------------------------------------
  public String toString()
  {
    String result = super.toString();
    result += "\nCurrent hours: " + hoursWorked;
    return result;
  }
}
```

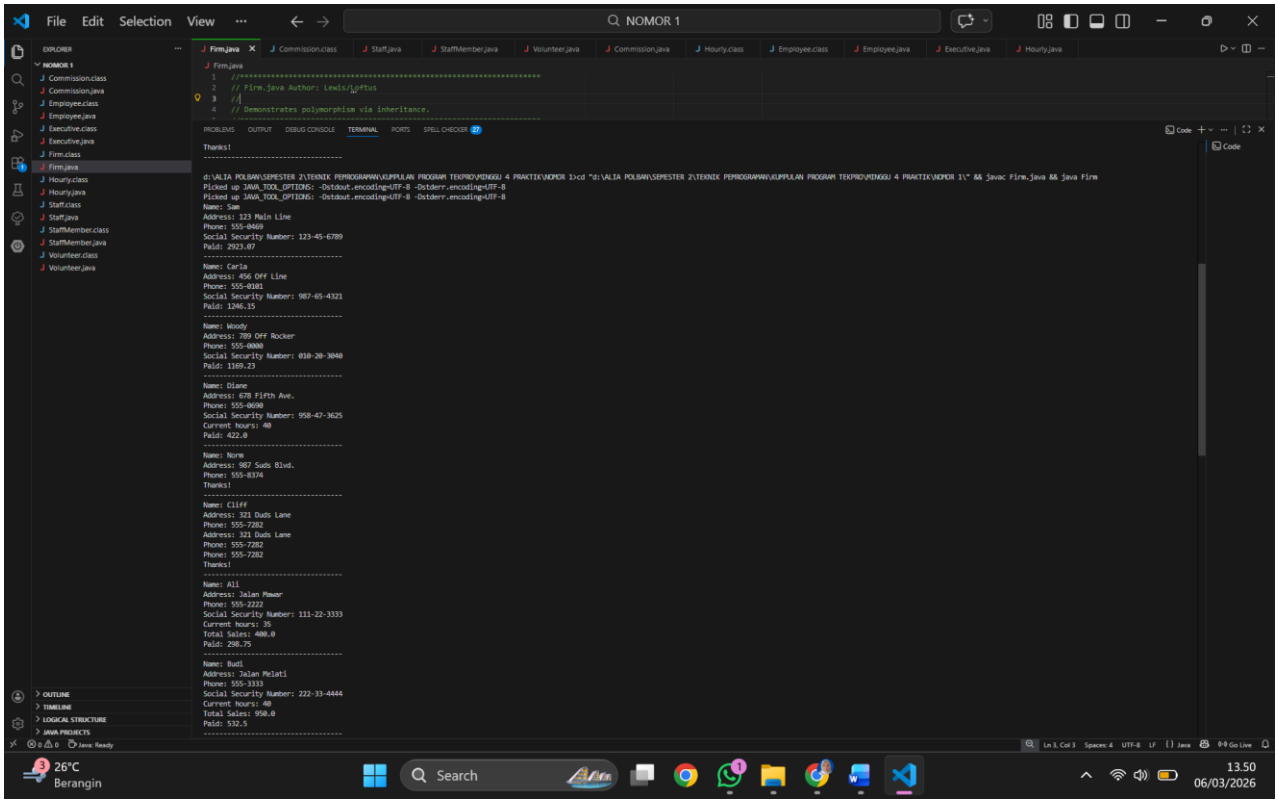
Commission.java

```
public class Commission extends Hourly { // 1. Membuat class Commision
                                         // 2. Class extends Hourly
  private double totalSales; //3. Variabel Total Sales
  private double commissionRate; //Variabel Commision Rate

  //4. Constructor dengan 6 Parameter
  public Commission(String eName, String eAddress, String ePhone,
        String socSecNumber, double rate,
        double commissionRate)
  {
    //5. Constructor Memanggil Parent (super)
    super(eName, eAddress, ePhone, socSecNumber, rate);
    this.commissionRate = commissionRate;
    totalSales = 0;
  }
  //6. Menambahkan Method addSales()
  public void addSales(double sales)
  {
    totalSales += sales;
  }

  //7. Menambahkan Method pay()
```

```java
  public double pay()
  {
    double payment = super.pay();
    payment += totalSales * commissionRate;
    //8. Reset totalSales
    totalSales = 0;

    return payment;
  }
  public String toString()
  {
    //9. Reset totalSales
    String result = super.toString();
    result += "\nTotal Sales: " + totalSales;
    return result;
  }
}
```

Screenshot Hasil





Penjelasan Permasalahan dan Solusi

Pada program telah disediakan sistem pengelolaan pegawai yang terdiri dari berbagai jenis, yaitu Executive, Employee, Hourly, dan Volunteer. Program ini juga dirancang menggunakan konsep pewarisan dan polimorfisme. Setiap pegawai memiliki cara hitung gaji yang berbeda, tetapi semuanya bersumber dari kelas yang sama yaitu StaffMember.

Permasalahan pada program ini adalah menambahkan jenis pegawai baru yang dibayar berdasarkan jam kerja sekaligus memperoleh komisi dari penjualan.

Solusi yang dilakukan adalah dengan membuat kelas baru bernama Commission yang merupakan turunan dari kelas Hourly. Pada kelas ini ditambahkan dua variabel baru yaitu totalSales untuk menyimpan total penjualan dan commissionRate untuk menyimpan persentase komisi. Selain itu, dibuat juga metode addSales() untuk menambahkan nilai penjualan dan metode pay() yang menghitung total gaji dengan menjumlahkan gaji berdasarkan jam kerja dengan komisi dari penjualan. Selanjutnya dilakukan perubahan pada Staff.java dengan menambah ukuran array menjadi 8 dan menambahkan dua pegawai Commission beserta data jam kerja dan total penjualannya.

Setelah semua perubahan dilakukan, program kemudian dikompilasi dan dijalankan untuk memastikan bahwa kelas baru dapat bekerja dengan baik bersama kelas-kelas lainnya. Hasil output menunjukkan bahwa sistem dapat menghitung gaji setiap jenis pegawai dengan benar, termasuk pegawai Commission yang menerima gaji berdasarkan jam kerja serta komisi dari penjualan.

Nama Teman Hal yang Dibantu (Opsional)

-

No. 2 Inheritance via Polymorphism (Painting Shapes)

Soal Praktikum
Silahkan membaca soal pada file lampiran: 2-Polymorphism via Inheritance.pdf
Source Code

Cylinder.java

```
// 4. Membuat class Cylinder yang merupakan turunan dari Shape
public class Cylinder extends Shape
{
   private double radius;
   private double height;

   public Cylinder(double r, double h)
   {
      super("Cylinder");
      radius = r;
      height = h;
   }

   public double area()
   {
      return Math.PI * radius * radius * height;
   }

   public String toString()
   {
      return super.toString() + " of radius " + radius + " and height " + height;
   }
}
```

Paint.java

```
//*******************************************************
// Paint.java
//
// Represents a type of paint that has a fixed area
// covered by a gallon. All measurements are in feet.
//*******************************************************
public class Paint {
   private double coverage; //number of square feet per gallon
   //----------------------------------------
   // Constructor: Sets up the paint object.
   //----------------------------------------
   public Paint(double c) {
      coverage = c;
   }
   //-------------------------------------------------
   // Returns the amount of paint (number of gallons)
   // needed to paint the shape given as the parameter.
   //-------------------------------------------------
```

```
  public double amount(Shape s) { // 5. Memperbaiki method amount() pada class Paint
    System.out.println("Computing amount for " + s);
    return s.area() / coverage;
  }
}
```

PaintThings.java

```
//*************************************************************
// PaintThings.java
//
// Computes the amount of paint needed to paint various
// things. Uses the amount method of the paint class which
// takes any Shape as a parameter.
//*************************************************************
import java.text.DecimalFormat;
  public class PaintThings {
    //----------------------------------------
    // Creates some shapes and a Paint object
    // and prints the amount of paint needed
    // to paint each shape.
    //----------------------------------------
    public static void main (String[] args) {
      final double COVERAGE = 350;
      Paint paint = new Paint(COVERAGE);

      // 6. Menginstansiasi objek pada class PaintThings
      Rectangle deck = new Rectangle(20, 35);
      Sphere bigBall = new Sphere(15);
      Cylinder tank = new Cylinder(10, 30);
      double deckAmt, ballAmt, tankAmt;

      // 7. Menghitung jumlah cat yang dibutuhkan
      deckAmt = paint.amount(deck);
      ballAmt = paint.amount(bigBall);
      tankAmt = paint.amount(tank);

      DecimalFormat fmt = new DecimalFormat("0.#");
      //8. Menampilkan hasil perhitungan jumlah cat
      System.out.println ("\nNumber of gallons of paint needed...");
      System.out.println ("Deck " + fmt.format(deckAmt));
      System.out.println ("Big Ball " + fmt.format(ballAmt));
      System.out.println ("Tank " + fmt.format(tankAmt));
    }
  }
```

Rectangle.java

```
// 3. Membuat class Rectangle yang merupakan turunan dari Shape
public class Rectangle extends Shape
{
```

```
    private double length;
    private double width;

    public Rectangle(double l, double w) {
        super("Rectangle");
        length = l;
        width = w;
    }

    public double area() {
        return length * width;
    }

    public String toString() {
        return super.toString() + " of length " + length + " and width " + width;
    }
}
```

Shape.java

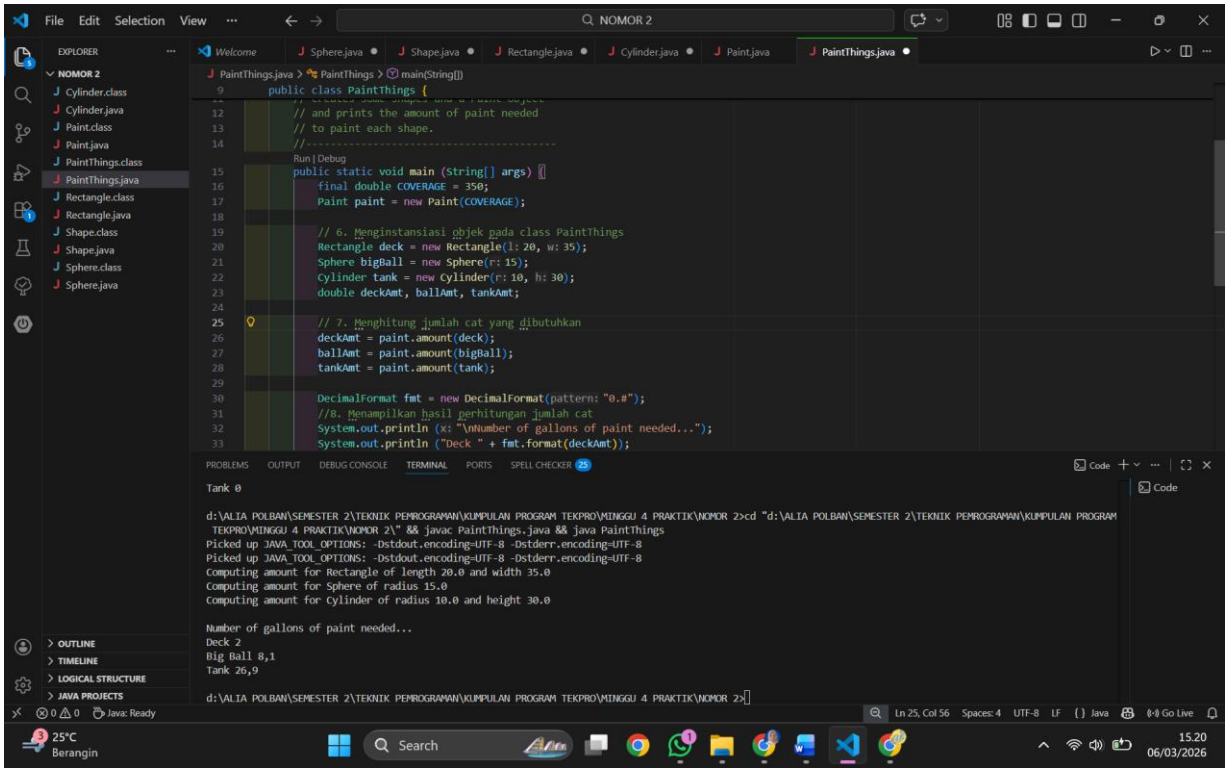
```
//1. Membuat Abstract Class Shape
public abstract class Shape {
    protected String shapeName;

    public Shape(String name) {
        shapeName = name;
    }

    public abstract double area();

    public String toString() {
        return shapeName;
    }
}
```

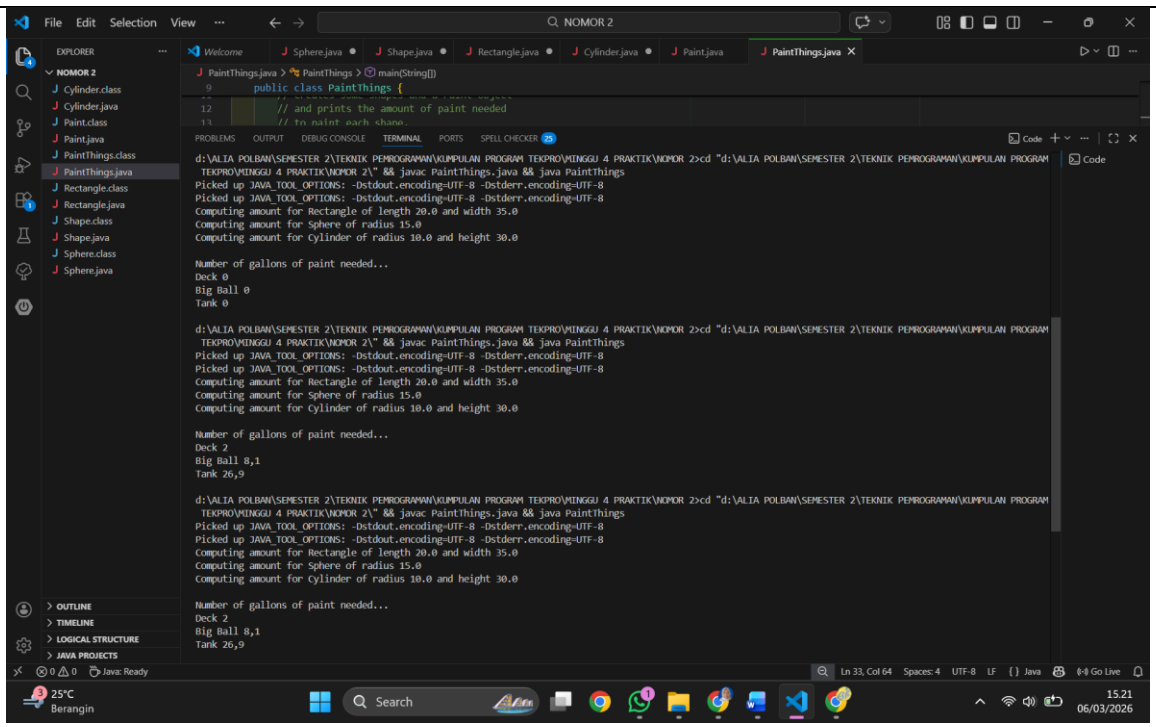
Sphere.java

```
//2. Membuat class Sphere yang merupakan turunan (extends) dari Shape
public class Sphere extends Shape {
    private double radius; //radius in feet

    //-----------------------------------
    // Constructor: Sets up the sphere.
    //-----------------------------------
    public Sphere(double r) {
        super("Sphere");
        radius = r;
    }

    //----------------------------------------
    // Returns the surface area of the sphere.
```

```
  //----------------------------------------
  public double area() {
    return 4 * Math.PI * radius * radius;
  }

  //-----------------------------------
  // Returns the sphere as a String.
  //-----------------------------------
  public String toString() {
    return super.toString() + " of radius " + radius;
  }
}
```

Screenshot Hasil





Penjelasan Permasalahan dan Solusi

Pada program telah disediakan sistem perhitungan kebutuhan cat untuk berbagai bentuk objek menggunakan konsep pemrograman berorientasi objek. Program ini dirancang menggunakan konsep **pewarisan (inheritance)** dan **polimorfisme (polymorphism)** dengan sebuah kelas induk yaitu Shape. Kelas ini kemudian memiliki beberapa kelas turunan yaitu Sphere, Rectangle, dan Cylinder. Setiap bentuk memiliki cara perhitungan luas yang berbeda, namun semuanya menggunakan struktur kelas yang sama.

Permasalahan pada program ini adalah membuat sistem yang dapat menghitung jumlah cat yang dibutuhkan untuk mengecat berbagai bentuk objek yang berbeda dengan menggunakan satu metode yang sama.

Solusi yang dilakukan adalah dengan membuat kelas abstrak Shape yang memiliki atribut shapeName serta metode abstrak area(). Metode ini kemudian diimplementasikan pada setiap kelas turunan yaitu Rectangle, Sphere, dan Cylinder sesuai dengan rumus luas masing-masing bentuk. Selain itu dibuat juga kelas Paint yang memiliki atribut coverage yang menyatakan luas area yang dapat dicat oleh satu gallon cat. Pada kelas ini terdapat metode amount() yang berfungsi menghitung jumlah cat yang dibutuhkan dengan membagi luas objek dengan nilai coverage.

Selanjutnya pada program PaintThings, dibuat beberapa objek bentuk yaitu deck sebagai Rectangle, bigBall sebagai Sphere, dan tank sebagai Cylinder. Program kemudian memanggil metode amount() dari kelas Paint untuk menghitung kebutuhan cat pada setiap objek.

Setelah semua bagian program dibuat, program dikompilasi dan dijalankan untuk memastikan bahwa perhitungan kebutuhan cat dapat berjalan dengan benar. Hasil output menunjukkan jumlah gallon cat yang diperlukan untuk masing-masing objek, sekaligus

menunjukkan penerapan konsep **polymorphism**, dimana metode amount() dapat menerima berbagai objek turunan dari kelas Shape.
Nama Teman Hal yang Dibantu (Opsional)
-

No. 3 Polymorphic Sorting

Soal Praktikum
Silahkan membaca soal pada file lampiran: 3-Polymorphic Sorting.pdf
Source Code

Numbers.java

```
// ******************************************************
// Numbers.java
// Demonstrates selectionSort on an array of integers.
// ******************************************************
import java.util.Scanner;
public class Numbers {
   // --------------------------------------------
   // Reads in an array of integers, sorts them,
   // then prints them in sorted order.
   // --------------------------------------------
   public static void main (String[] args) {

      Integer[] intList;
      int size;

      Scanner scan = new Scanner(System.in);
      System.out.print ("\nHow many integers do you want to sort? ");
      size = scan.nextInt();
      intList = new Integer[size];
      System.out.println ("\nEnter the numbers...");

      for (int i = 0; i < size; i++)
         intList[i] = scan.nextInt();

      Sorting.insertionSort(intList);
      System.out.println ("\nYour numbers in sorted order...");

      for (int i = 0; i < size; i++)
         System.out.print(intList[i] + " ");

      System.out.println ();
   }
}
```

Salesperson.java

```
// ******************************************************
// Salesperson.java
// Represents a sales person who has a first name, last
// name, and total number of sales.
// ******************************************************
public class Salesperson implements Comparable {
   private String firstName, lastName;
   private int totalSales;
   //-------------------------------------------------------
   // Constructor: Sets up the sales person object with
   // the given data.
   //-------------------------------------------------------
   public Salesperson (String first, String last, int sales) {
      firstName = first;
      lastName = last;
      totalSales = sales;
   }
   //-----------------------------------------
   // Returns the sales person as a string.
   //-----------------------------------------
   public String toString() {
      return lastName + ", " + firstName + ": \t" + totalSales;
   }
   //-----------------------------------------
   // Returns true if the sales people have
   // the same name.
   //-----------------------------------------
   public boolean equals (Object other) {
      return (lastName.equals(((Salesperson)other).getLastName()) &&
      firstName.equals(((Salesperson)other).getFirstName()));
   }
   //-------------------------------------------------
   // Order is based on total sales with the name
   // (last, then first) breaking a tie.
   //-------------------------------------------------
   public int compareTo(Object other) {
      Salesperson otherPerson = (Salesperson) other;
      int result = otherPerson.totalSales - this.totalSales;
      if(result == 0) {
         result = otherPerson.lastName.compareTo(this.lastName);
         if(result == 0)
            result = otherPerson.firstName.compareTo(this.firstName);
      }
      return result;
   }
   //-------------------------
   // First name accessor.
   //-------------------------
```

```
  public String getFirstName() {
    return firstName;
  }
  //-------------------------
  // Last name accessor.
  //-------------------------
  public String getLastName() {
    return lastName;
  }
  //-------------------------
  // Total sales accessor.
  //-------------------------
  public int getSales() {
    return totalSales;
  }
}
```

Sorting.java

```
//*******************************************************************
// Sorting.java Author: Lewis/Loftus
// Demonstrates the selection sort and insertion sort algorithms.
//*******************************************************************
public class Sorting {
  //-----------------------------------------------------------------
  // Sorts the specified array of objects using the selection
  // sort algorithm.
  //-----------------------------------------------------------------
  public static void selectionSort (Comparable[] list) {
    int min;
    Comparable temp;

    for (int index = 0; index < list.length-1; index++) {
      min = index;

      for (int scan = index+1; scan < list.length; scan++)
        if (list[scan].compareTo(list[min]) < 0)
          min = scan;

      temp = list[min];
      list[min] = list[index];
      list[index] = temp;
    }
  }
  //-----------------------------------------------------------------
  // Sorts the specified array of objects using the insertion
  // sort algorithm.
  //-----------------------------------------------------------------
```

```
  public static void insertionSort (Comparable[] list) {
    for (int index = 1; index < list.length; index++) {
      Comparable key = list[index];
      int position = index;

      // descending
      while (position > 0 && key.compareTo(list[position-1]) > 0) {
        list[position] = list[position-1];
        position--;
      }
      list[position] = key;
    }
  }
}
```

Strings.java

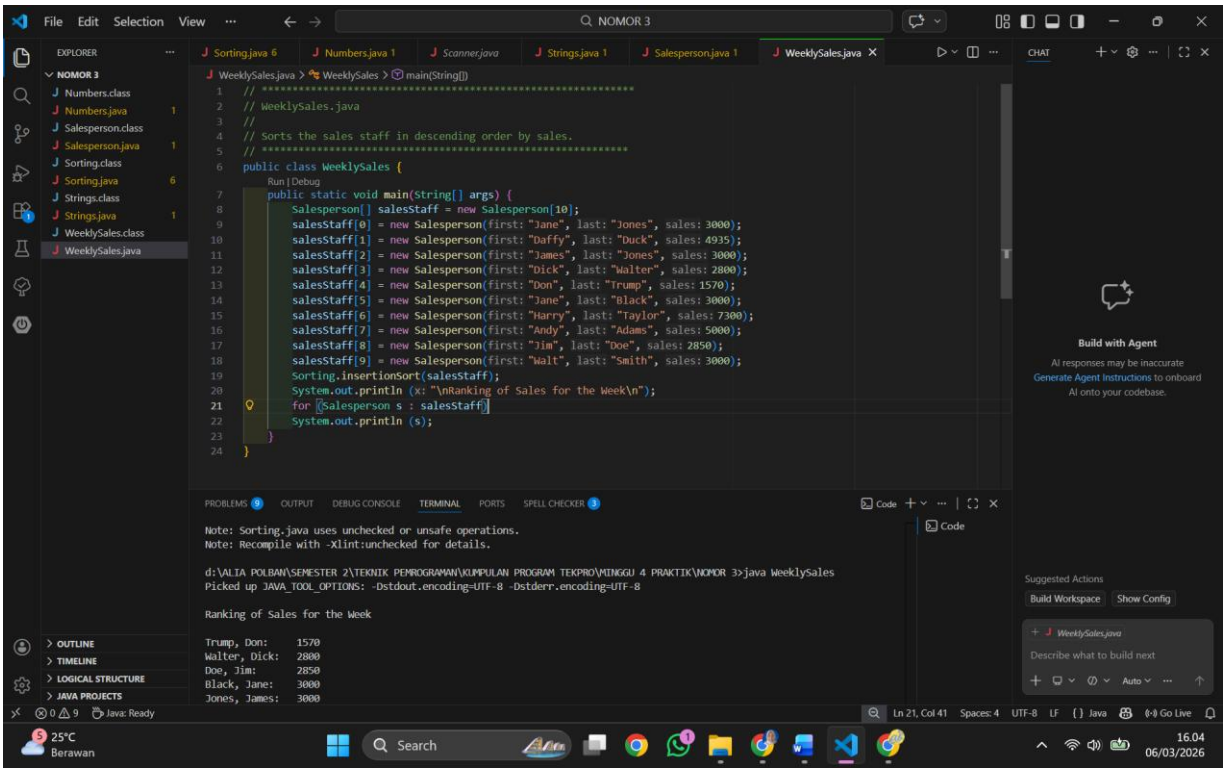
```
//2. Membuat Strings Java
import java.util.Scanner;

public class Strings{
  public static void main(String[] args) {
    String[] wordList;
    int size;
    Scanner scan = new Scanner(System.in);
    System.out.print("How many words do you want to sort? ");
    size = scan.nextInt();
    wordList = new String[size];

    System.out.println("Enter the words...");
    for(int i = 0; i < size; i++)
      wordList[i] = scan.next();
    Sorting.insertionSort(wordList); //4. Mengganti method sorting
    System.out.println("\nWords in sorted order:");

    for(String word : wordList)
      System.out.print(word + " ");
    System.out.println();
  }
}
```

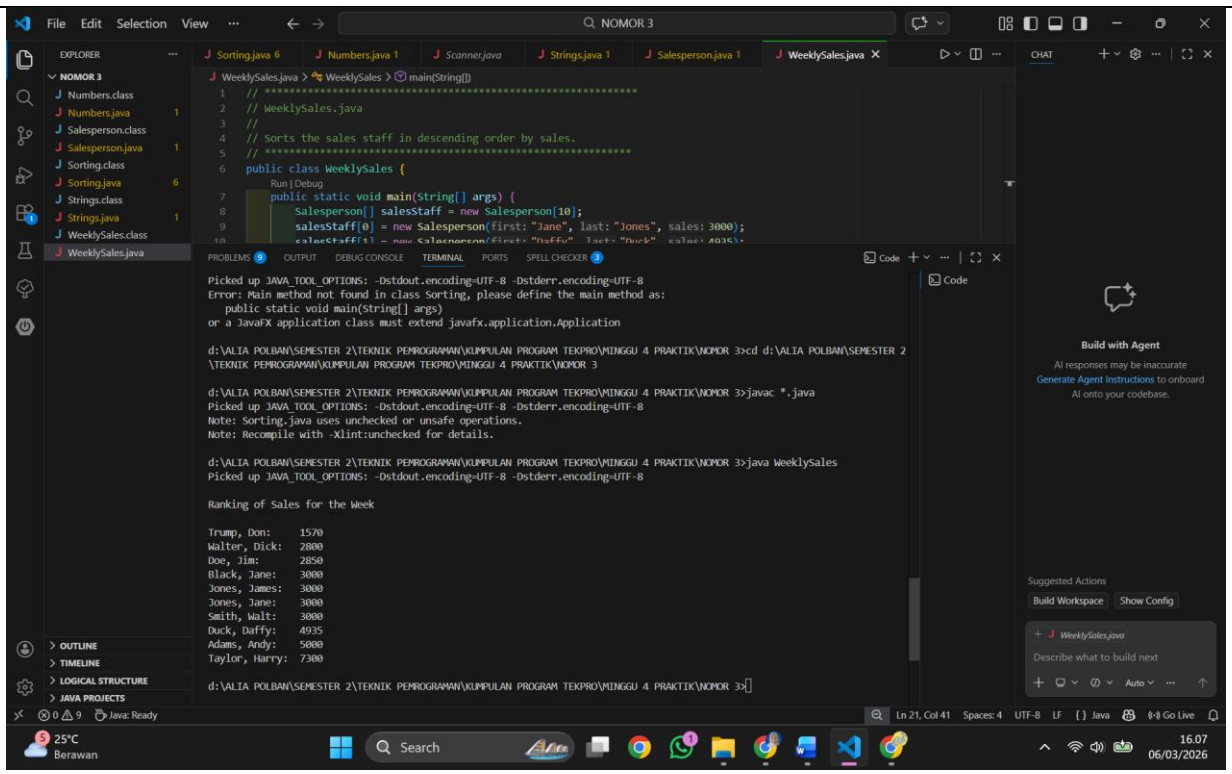
WeeklySales.java

```
// ************************************************************
// WeeklySales.java
// Sorts the sales staff in descending order by sales.
// ***********************************************************
public class WeeklySales {
  public static void main(String[] args) {
    Salesperson[] salesStaff = new Salesperson[10];
    salesStaff[0] = new Salesperson("Jane", "Jones", 3000);
```

```
    salesStaff[1] = new Salesperson("Daffy", "Duck", 4935);
    salesStaff[2] = new Salesperson("James", "Jones", 3000);
    salesStaff[3] = new Salesperson("Dick", "Walter", 2800);
    salesStaff[4] = new Salesperson("Don", "Trump", 1570);
    salesStaff[5] = new Salesperson("Jane", "Black", 3000);
    salesStaff[6] = new Salesperson("Harry", "Taylor", 7300);
    salesStaff[7] = new Salesperson("Andy", "Adams", 5000);
    salesStaff[8] = new Salesperson("Jim", "Doe", 2850);
    salesStaff[9] = new Salesperson("Walt", "Smith", 3000);
    Sorting.insertionSort(salesStaff);
    System.out.println ("\nRanking of Sales for the Week\n");
    for (Salesperson s : salesStaff)
    System.out.println (s);
  }
}
```

Screenshot Hasil





Penjelasan Permasalahan dan Solusi

Pada program telah disediakan sistem pengurutan data menggunakan algoritma sorting dengan memanfaatkan konsep pemrograman berorientasi objek. Program ini terdiri dari beberapa kelas yaitu Sorting, Numbers, Strings, Salesperson, dan WeeklySales. Kelas Sorting menyediakan metode pengurutan seperti *selection sort* dan *insertion sort* yang dapat digunakan untuk mengurutkan array objek.

Permasalahan pada program ini adalah membuat sistem pengurutan yang dapat digunakan untuk berbagai jenis objek yang berbeda menggunakan satu algoritma sorting yang sama. Selain itu, pada objek Salesperson diperlukan aturan pengurutan khusus yaitu berdasarkan total penjualan dan jika terdapat nilai penjualan yang sama maka pengurutan ditentukan berdasarkan nama belakang dan nama depan.

Solusi yang dilakukan adalah dengan menggunakan interface Comparable dan mengimplementasikan metode compareTo() pada kelas Salesperson. Metode ini digunakan untuk membandingkan dua objek Salesperson berdasarkan total penjualan, dan jika nilainya sama maka dibandingkan berdasarkan nama belakang dan nama depan. Kelas Sorting menggunakan parameter bertipe Comparable[] sehingga metode pengurutan dapat digunakan untuk berbagai jenis objek. Selanjutnya pada program WeeklySales, dibuat beberapa objek Salesperson yang disimpan dalam sebuah array dan kemudian diurutkan menggunakan metode insertionSort() dari kelas Sorting.

Setelah program dikompilasi dan dijalankan, hasil output menampilkan peringkat penjualan mingguan dari tertinggi hingga terendah, sekaligus menunjukkan penerapan polymorphism, dimana satu metode sorting dapat digunakan untuk mengurutkan berbagai jenis objek.

Nama Teman Hal yang Dibantu (Opsional)

-